

Menguasai Fundamental Go: Setup Lingkungan, Manajemen Variabel, dan Operasi Data

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Selamat datang di topik fundamental Go! Pada modul ini, kita tidak hanya akan mempelajari cara menulis kode dasar, tetapi juga memahami konsep di balik layar (*under the hood*) bagaimana Go mengelola memori, variabel, dan operasi data. Modul ini dirancang agar Anda memiliki fondasi yang kuat untuk menulis kode Go yang efisien, aman, dan standar industri (*production-ready*).

Sesi 1: Setup Lingkungan & Best Practices Go (30 Menit)

Sebelum memulai pemrograman, pastikan Anda memahami bagaimana Go mengelola proyek dan dependensinya.

1. Go Environment Variables

Go memiliki beberapa variabel lingkungan (*environment variables*) penting yang mengatur cara compiler bekerja:

- **GOROOT** : Lokasi di mana SDK/compiler Go diinstal di komputer Anda. Biasanya ini diatur secara otomatis saat instalasi.
- **GOPATH** : Direktori kerja Go sebelum era Go Modules (Go < 1.11). Saat ini, **GOPATH** utamanya digunakan untuk menyimpan modul pihak ketiga yang diunduh (`$GOPATH/pkg/mod`) dan file *binary* hasil instalasi tools (`$GOPATH/bin`).
- **GOBIN** : Direktori tempat Go menaruh file executable hasil perintah `go install` . Agar tools CLI Go bisa diakses global, pastikan `$GOBIN` atau `$GOPATH/bin` terdaftar di sistem `PATH` OS Anda.

2. Inisialisasi Proyek dengan Go Modules

Sejak Go 1.11, kita wajib menggunakan **Go Modules** untuk manajemen dependensi. Hindari membuat kode Go di luar modul karena akan menyulitkan import package.

Langkah inisialisasi modul:

```
# Buat direktori kerja baru
mkdir go-fundamental
cd go-fundamental

# Inisialisasi Go Module dengan nama modul (biasanya path repository git)
go mod init github.com/username/go-fundamental
```

Perintah ini menghasilkan file `go.mod` yang berfungsi seperti `package.json` di Node.js atau `requirements.txt` / `pyproject.toml` di Python.

3. Setup VS Code & Language Server (gopls)

Untuk produktivitas terbaik di VS Code:

1. Install ekstensi **Go** resmi dari Google.
2. Buka command palette (`Ctrl+Shift+P`), ketik `Go: Install/Update Tools` .
3. Pilih semua tools (termasuk `gopls` sebagai Language Server, `dlv` untuk debugging, dan `golangci-lint` untuk static analysis), lalu klik **OK**.

Sesi 2: Manajemen Variabel & Tipe Data Tingkat Lanjut (60 Menit)

Go adalah bahasa pemrograman yang bertipe data statis (**Statically Typed**). Artinya, tipe data dari sebuah variabel harus diketahui saat waktu kompilasi (*compile time*), dan tidak dapat berubah selama program berjalan.

1. Deklarasi Variabel & Type Inference

Ada beberapa cara mendeklarasikan variabel di Go, masing-masing dengan kegunaan yang berbeda:

A. Menggunakan Keyword `var` (Dengan/Tanpa Tipe Data)

```
// Eksplisit dengan inisialisasi nilai
var email string = "developer@go.dev"

// Eksplisit tanpa nilai (akan mendapatkan Zero Value)
var age int

// Implisit (Type Inference) - Go menebak tipe data berdasarkan nilainya
var isManager = true // tipe data boolean
```

B. Menggunakan Short Declaration (`:=`)

Operator `:=` digunakan untuk mendeklarasikan sekaligus menginisialisasi variabel dengan cara yang lebih ringkas.

```
name := "John Doe" // Otomatis bertipe string
score := 95.5       // Otomatis bertipe float64
```

IMPORTANT

Aturan Penting `:=` :

1. Hanya bisa digunakan di **dalam fungsi** (lokal). Untuk variabel tingkat package (global), wajib menggunakan keyword `var` .
2. Harus ada setidaknya satu variabel baru di sebelah kiri operator jika mendeklarasikan multi-variabel.

C. Deklarasi Multi-Variabel

```
var x, y, z int = 1, 2, 3
title, isPublished := "Golang Basics", false
```

2. Zero Value

Jika variabel dideklarasikan tanpa nilai awal, Go akan otomatis memberikan nilai bawaan (**Zero Value**). Hal ini mencegah perilaku acak akibat membaca memori sampah.

Tipe Data	Zero Value
<code>int</code> , <code>float</code> , <code>byte</code> , <code>rune</code>	<code>0</code> atau <code>0.0</code>
<code>string</code>	<code>""</code> (string kosong)
<code>boolean</code>	<code>false</code>
Pointer, Slice, Map, Interface, Function, Channel	<code>nil</code>

3. Konstanta & `iota` (Enum ala Go)

Konstanta dideklarasikan menggunakan keyword `const` . Nilainya harus sudah diketahui saat kompilasi dan bersifat *read-only*.

Go tidak memiliki keyword khusus untuk `enum`, tetapi kita menggunakan kombinasi `const` dan identifier `iota` untuk membuat enum yang bersih dan otomatis bertambah (*auto-increment*):

```
package main

import "fmt"

// Menentukan level otorisasi user menggunakan iota
const (
    Guest int = iota // bernilai 0
    User             // bernilai 1 (otomatis)
    Admin            // bernilai 2 (otomatis)
)

func main() {
    fmt.Printf("Role Guest: %d, User: %d, Admin: %d\n", Guest, User, Admin)
}
```

4. Tipe Data Dasar & Detail Memori

Sebagai Backend Engineer, pemilihan ukuran tipe data yang tepat sangat penting untuk efisiensi memori:

- **Integer:**
 - *Signed* (Bisa negatif & positif): `int8` (-128 s.d 127), `int16`, `int32` (disebut juga `rune` untuk karakter Unicode), `int64`, dan `int` (ukurannya menyesuaikan arsitektur CPU, yaitu 32-bit atau 64-bit).
 - *Unsigned* (Hanya positif): `uint8` (disebut juga `byte` dengan jangkauan 0 s.d 255), `uint16`, `uint32`, `uint64`, dan `uint`.
- **Floating Point:**
 - `float32` dan `float64` (default untuk desimal). Gunakan `float64` untuk perhitungan presisi tinggi.
- **String:**
 - Bersifat **immutable** (tidak dapat diubah setelah dibuat).
 - Dinyatakan dengan *double quotes* `"..."` (mendukung escape character seperti `\n`) atau *backtick* ``...`` (raw string literal, cocok untuk teks multiline atau query SQL).

5. Konversi Tipe Data (Type Casting) & Library `strconv`

Go sangat ketat dan tidak melakukan konversi tipe data implisit (tidak ada *implicit casting*). Anda tidak bisa menjumlahkan `int` dengan `int32` secara langsung tanpa konversi.

```
var a int = 10
var b int32 = 20

// Salah: total := a + b (Compile Error!)
// Benar:
total := a + int(b)
```

Untuk konversi string ke angka (atau sebaliknya), gunakan package bawaan `strconv` :

```
package main

import (
    "fmt"
    "strconv"
)

func main() {
    // Konversi Angka ke String (Integer to ASCII)
    strAge := strconv.Itoa(25)

    // Konversi String ke Angka (ASCII to Integer)
    // Fungsi ini mengembalikan 2 nilai: (hasil_konversi, error)
    age, err := strconv.Atoi("30")
    if err != nil {
        fmt.Println("Error konversi:", err)
        return
    }

    fmt.Println("String Age:", strAge)
    fmt.Println("Int Age:", age)
}
```

6. Variable Shadowing (Gotcha Level Intermediate)

Variable Shadowing terjadi ketika sebuah variabel dideklarasikan ulang dengan nama yang sama di dalam cakupan (*scope*) yang lebih dalam (seperti blok `if`, `for`, atau fungsi lokal). Ini sering menjadi penyebab bug tersembunyi.

```
package main

import "fmt"

func main() {
    token := "token_utama"
    isValid := true

    if isValid {
        // Menggunakan := di sini menciptakan variabel 'token' BARU
        // yang hanya hidup di dalam blok 'if' ini.
        token := "token_rahasia"
        fmt.Println("Di dalam block:", token) // Output: token_rahasia
    }

    // Variabel token utama di luar tidak berubah
    fmt.Println("Di luar block:", token) // Output: token_utama
}
```

TIP

Best Practice: Berhati-hatilah saat menggunakan operator `:=` di dalam blok percabangan/perulangan jika Anda ingin mengubah nilai dari variabel yang berada di scope luar. Gunakan operator assignment biasa `=` jika variabelnya sudah dideklarasikan sebelumnya.

Sesi 3: Operasi Data & Operator (45 Menit)

Setelah data disimpan di dalam variabel, kita perlu memprosesnya menggunakan operator.

1. Operator Aritmatika

Operator standar untuk perhitungan matematika: `+` (penjumlahan/gabung string), `-` (pengurangan), `*` (perkalian), `/` (pembagian), `%` (modulus/sisa bagi).

WARNING

Penting Mengenai Increment (`++`) dan Decrement (`--`): Di Go, `++` dan `--` adalah sebuah **statement**, bukan *expression* atau *operator*. Artinya:

- Anda hanya bisa menulis: `x++` atau `x--` .
- Anda **tidak bisa** menulis: `y = x++` atau `fmt.Println(x++)` karena operasi tersebut tidak mengembalikan nilai.
- Tidak ada pre-increment seperti `++x` .

2. Operator Perbandingan

Digunakan untuk membandingkan dua nilai dan menghasilkan nilai boolean (`true` atau `false`): `=` , `≠` , `<` , `>` , `≤` , `≥` . *Catatan: Kedua nilai yang dibandingkan harus memiliki tipe data yang identik.*

3. Operator Logika

Digunakan untuk mengevaluasi ekspresi logika majemuk:

- `&&` (Logical AND): Menghasilkan `true` jika kedua kondisi bernilai benar.
- `||` (Logical OR): Menghasilkan `true` jika salah satu kondisi bernilai benar.
- `!` (Logical NOT): Membalikkan nilai boolean.

Short-Circuit Evaluation

Go menggunakan evaluasi lintas cepat (*short-circuit evaluation*) untuk operator logika:

- Pada ekspresi `A && B` , jika `A` bernilai `false` , maka `B` **tidak akan dievaluasi** karena hasilnya sudah pasti `false` .
- Pada ekspresi `A || B` , jika `A` bernilai `true` , maka `B` **tidak akan dievaluasi** karena hasilnya sudah pasti `true` .

Sesi 4: Praktikum Mandiri: "Sistem Kasir & Kalkulator Pajak Toko" (45 Menit)

Sekarang kita akan menggabungkan semua materi di atas ke dalam sebuah proyek praktikum riil. Kita akan membangun program CLI kasir interaktif untuk menghitung total belanjaan, diskon member, pajak pertambahan nilai (PPN), dan mencetak struk belanja terformat.

1. Target Proyek

1. Menerima input nama pelanggan, harga barang, jumlah barang, dan status keanggotaan (member).
2. Melakukan konversi dan kalkulasi data keuangan dengan presisi menggunakan `float64`.
3. Menentukan diskon berdasarkan kombinasi keanggotaan dan total belanja menggunakan operator perbandingan dan logika.
4. Menghitung PPN sebesar 11% dari total setelah diskon.
5. Mencetak struk transaksi dengan format rapi yang presisi menggunakan formatting verbs.

2. Struktur Proyek

Silakan buat file `main.go` di direktori proyek Anda dan tuliskan kode berikut.


```

package main

import (
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
)

// Menggunakan iota untuk mendefinisikan persentase diskon berdasarkan level member
const (
    DiscountNonMember float64 = 0.0 // 0%
    DiscountMember    float64 = 0.05 // 5%
    DiscountPremium   float64 = 0.10 // 10%
)

func main() {
    // Scanner untuk membaca input teks yang mengandung spasi
    reader := bufio.NewReader(os.Stdin)

    fmt.Println("=====")
    fmt.Println("    APLIKASI KASIR TOKO - GOLANG BASE    ")
    fmt.Println("=====")

    // 1. Input Nama Pelanggan
    fmt.Print("Masukkan Nama Pelanggan : ")
    customerNameInput, _ := reader.ReadString('\n')
    customerName := strings.TrimSpace(customerNameInput)

    // 2. Input Nama Barang
    fmt.Print("Masukkan Nama Barang : ")
    itemNameInput, _ := reader.ReadString('\n')
    itemName := strings.TrimSpace(itemNameInput)

    // 3. Input Harga Barang (dan konversi string → float64)
    fmt.Print("Masukkan Harga Barang : ")
    priceInput, _ := reader.ReadString('\n')
    priceInput = strings.TrimSpace(priceInput)
    price, err := strconv.ParseFloat(priceInput, 64)
    if err != nil || price <= 0 {
        fmt.Println("Error: Input harga barang harus berupa angka positif!")
        return
    }

    // 4. Input Jumlah Barang (dan konversi string → int)
    fmt.Print("Masukkan Jumlah Barang : ")
    quantityInput, _ := reader.ReadString('\n')
    quantityInput = strings.TrimSpace(quantityInput)
    quantity, err := strconv.Atoi(quantityInput)
    if err != nil || quantity <= 0 {
        fmt.Println("Error: Input jumlah barang harus berupa angka bulat positif")
    }
}

```

```

        return
    }

    // 5. Input Status Member (Non-Member / Member / Premium)
    fmt.Println("Status Member (none/member/premium): ")
    memberInput, _ := reader.ReadString('\n')
    memberStatus := strings.ToLower(strings.TrimSpace(memberInput))

    // --- PROSES KALKULASI & OPERASI DATA ---

    // Hitung subtotal kotor
    subtotal := price * float64(quantity)

    // Tentukan rate diskon awal berdasarkan status member
    var discountRate float64
    switch memberStatus {
    case "member":
        discountRate = DiscountMember
    case "premium":
        discountRate = DiscountPremium
    default:
        discountRate = DiscountNonMember
    }

    // Logika Bisnis Tambahan (Operator Logika & Perbandingan):
    // Jika subtotal belanja lebih dari Rp 500.000, berikan tambahan diskon 2%
    // tanpa memandang status keanggotaan.
    hasExtraDiscount := subtotal > 500000.0
    if hasExtraDiscount {
        discountRate += 0.02
    }

    // Hitung nilai diskon
    discountAmount := subtotal * discountRate
    totalAfterDiscount := subtotal - discountAmount

    // Hitung PPN 11% dari total setelah diskon
    const taxRate = 0.11
    taxAmount := totalAfterDiscount * taxRate

    // Total Akhir yang harus dibayar
    grandTotal := totalAfterDiscount + taxAmount

    // --- FORMATTING OUTPUT (STRUK BELANJA) ---
    fmt.Println("\n=====")
    fmt.Println("                STRUK PEMBAYARAN                ")
    fmt.Println("=====")
    fmt.Printf("Pelanggan      : %s\n", customerName)
    fmt.Printf("Status Member : %s\n", strings.ToUpper(memberStatus))
    if hasExtraDiscount {
        fmt.Println("Promo          : Ekstra Diskon 2% (Belanja > 500k)")
    } else {
        fmt.Println("Promo          : Tidak ada promo tambahan")
    }

```

```

    }
    fmt.Println("-----")
    fmt.Printf("%-20s x%-3d : Rp %12.2f\n", itemName, quantity, price)
    fmt.Println("-----")
    fmt.Printf("Subtotal                : Rp %12.2f\n", subtotal)
    fmt.Printf("Diskon (%.0f%%)                : Rp %12.2f\n", discountRate*100, discountAmount)
    fmt.Printf("Total Setelah Diskon : Rp %12.2f\n", totalAfterDiscount)
    fmt.Printf("PPN (11%)                  : Rp %12.2f\n", taxAmount)
    fmt.Println("-----")
    fmt.Printf("TOTAL AKHIR                : Rp %12.2f\n", grandTotal)
    fmt.Println("=====")
    fmt.Println("          Terima Kasih Telah Berbelanja!          ")
    fmt.Println("=====")
}

```

3. Cara Menjalankan Proyek

Buka terminal di direktori proyek Anda dan jalankan perintah:

```
go run main.go
```

Coba jalankan dengan inputan uji coba berikut:

```

Masukkan Nama Pelanggan : Budi Santoso
Masukkan Nama Barang    : Keyboard Mechanical
Masukkan Harga Barang   : 6000000
Masukkan Jumlah Barang  : 1
Status Member (none/member/premium): member

```

Pastikan hasil kalkulasi diskon (5% + 2% ekstra karena > 500k = 7%) dan PPN 11% dihitung dengan benar pada struk belanja Anda.

Sesi 5: Tantangan Mandiri (45 Menit)

Untuk menguji pemahaman Anda mengenai variabel, konversi tipe data, dan operator di Go, selesaikan tantangan mandiri berikut:

Tantangan: Fitur Pembayaran & Uang Kembali

Modifikasi kode praktikum `main.go` di atas untuk menambahkan fitur pembayaran tunai dan kalkulasi uang kembalian.

Spesifikasi Fitur Baru:

1. Setelah total akhir (grand total) dihitung dan struk ditampilkan, program harus meminta input nominal uang tunai yang dibayarkan oleh pelanggan (`cashPaid`).
2. Konversikan input tersebut menjadi `float64` secara aman.
3. Lakukan pemeriksaan menggunakan **operator perbandingan**:
 - Jika uang tunai **kurang** dari grand total, tampilkan pesan error: `"Transaksi Gagal: Uang tunai tidak cukup!"`
 - Jika uang tunai **cukup atau lebih**, hitung kembalian menggunakan **operator aritmatika** pengurangan.
4. Tampilkan nominal kembalian dengan format desimal dua angka di belakang koma.

Cobalah menulis kodenya secara mandiri terlebih dahulu untuk melatih insting pemrograman Anda!

Sesi 6: Soal Latihan Praktik (Mandiri)

Untuk menguji pemahaman secara mendalam, buatlah proyek mini/file baru untuk masing-masing soal latihan di bawah ini:

Soal 1: Kalkulator Indeks Massa Tubuh (BMI) Berpresisi

Deskripsi: Buatlah program CLI untuk mengukur Indeks Massa Tubuh (BMI) seseorang.

- **Input:** Nama (string), Berat Badan (kg - `float64`), dan Tinggi Badan (cm - `float64`).
- **Proses:**
 1. Konversikan tinggi badan dari centimeter (cm) ke meter (m).
 2. Hitung nilai BMI menggunakan rumus: `BMI = Berat_Badan / (Tinggi_Badan_Meter * Tinggi_Badan_Meter)` .
 3. Tentukan status berat badan berdasarkan aturan:
 - `Underweight` : BMI < 18.5
 - `Normal` : BMI antara 18.5 s.d 24.9
 - `Overweight` : BMI antara 25.0 s.d 29.9
 - `Obese` : BMI >= 30.0
- **Output:** Tampilkan nama, nilai BMI (dibulatkan 2 angka di belakang koma), dan status berat badannya menggunakan `fmt.Printf` .

Soal 2: Konverter Durasi Detik ke Jam-Menit-Detik

Deskripsi: Buatlah program yang mengonversi durasi waktu dalam satuan detik menjadi format waktu yang lebih mudah dibaca.

- **Input:** Jumlah total detik (integer, contoh: `9125`).
- **Proses:**
 1. Hitung jumlah **jam** (`total_detik / 3600`).

2. Hitung sisa detik setelah diambil jam, lalu hitung jumlah **menit** (`sisa_detik / 60`).
 3. Dapatkan **sisa detik** terakhir (`sisa_detik % 60`).
- **Output:** Tampilkan output dengan format: "9125 detik setara dengan: 2 jam, 32 menit, 5 detik."

Soal 3: Generator & Validator Voucher Promo

Deskripsi: Sebuah e-commerce ingin memvalidasi kelayakan pelanggan dalam mengklaim voucher "SUPERHEMAT" .

- **Input:** Tahun Bergabung Member (int), Total Belanja Awal (float64), dan Status Keanggotaan ("premium" atau "regular").
- **Kriteria Kelayakan (Gunakan Operator Logika):** Pelanggan berhak mendapatkan diskon tambahan jika memenuhi **salah satu** kondisi berikut:
 1. Merupakan member `premium` **DAN** sudah bergabung sebelum tahun 2024.
 2. Merupakan member `regular` **DAN** total belanja awal lebih dari Rp 1.000.000.
- **Output:** Tampilkan status kelayakan (Boolean: `true` atau `false`) dan total belanja akhir setelah dipotong diskon 15% jika mereka layak mendapatkan promo tersebut.

Soal 4: Simulasi Tarif Parkir Progresif

Deskripsi: Buatlah program kalkulator parkir bandara dengan tarif progresif.

- **Input:** Jenis Kendaraan ("mobil" atau "motor") dan Durasi Parkir (dalam jam - integer).
- **Aturan Tarif:**
 - **Mobil:** 1 jam pertama Rp 10.000, setiap jam berikutnya Rp 5.000.
 - **Motor:** 1 jam pertama Rp 5.000, setiap jam berikutnya Rp 2.000.
 - Jika durasi parkir lebih dari 24 jam, dikenakan biaya denda tambahan sebesar Rp 50.000 (Gunakan operator logika dan perbandingan).
- **Output:** Tampilkan rincian jenis kendaraan, durasi, dan total tarif akhir yang harus dibayar.

Soal 5: Konverter Suhu Multi-Skala

Deskripsi: Buatlah program untuk mengonversi suhu dari Celcius ke tiga skala suhu lainnya (Fahrenheit, Reamur, dan Kelvin).

- **Input:** Suhu dalam Celcius (float64).
- **Rumus Konversi:**
 - `Fahrenheit = (Celcius * 9/5) + 32` (Catatan: Hati-hati dengan pembagian bilangan bulat di Go!)
 - `Reamur = Celcius * 4/5`
 - `Kelvin = Celcius + 273.15`
- **Output:** Tampilkan hasil konversi lengkap dalam format tabel sederhana menggunakan *formatting space* pada `fmt.Printf` agar rapi dan lurus.